

COMPREHENSIVE TECHNICAL SPECIFICATIONS - WEEKOOK V2

Project: Weekook - Cuisine à Domicile Platform **Environment:** Development

Date: 2026-02-18

Tech Stack: React 18 + TypeScript + Vite 6 + Express.js + Prisma + MySQL

TABLE OF CONTENTS

- [1. PRISMA SCHEMA](#)
- [2. DATABASE MODELS](#)
- [3. BACKEND - SERVER ROUTES](#)
- [4. ZOD VALIDATION SCHEMAS](#)
- [5. MIDDLEWARE](#)
- [6. CONFIGURATION & UTILITIES](#)
- [7. EMAIL SYSTEM](#)
- [8. FRONTEND ROUTES](#)
- [9. API CLIENT & AUTH](#)
- [10. ENVIRONMENT VARIABLES](#)
- [11. PACKAGE DEPENDENCIES](#)

PRISMA SCHEMA

File: prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

// User model - All users (customers + kookers)
model User {
  id          Int      @id @default(autoincrement())
  email       String    @unique
  pendingEmail String?  @map("pending_email")      // Email en attente de
confirmation
  emailVerified Boolean  @default(true) @map("email_verified") // false après
inscription jusqu'à vérification
  password    String
  firstName   String    @map("first_name")
  lastName    String    @map("last_name")
  phone       String?
  avatar      String?
  role        String    @default("user") // "user", "kooker", "admin",
"suspended"
```

```

isAdmin          Boolean    @default(false) @map("is_admin")
createdAt        DateTime   @default(now()) @map("created_at")
updatedAt        DateTime   @updatedAt @map("updated_at")

kookerProfile    KookerProfile?
userProfile      UserProfile?
bookings         Booking[]   @relation("UserBookings")
reviews          Review[]
favorites        Favorite[]
sentMessages     Message[]   @relation("SentMessages")
receivedMessages Message[]   @relation("ReceivedMessages")
reviewsReceived  Review[]    @relation("ReviewsReceived")
passwordResetTokens PasswordResetToken[]

@@map("users")
}

// Kooker professional profile
model KookerProfile {
  id          Int          @id @default(autoincrement())
  userId      Int          @unique @map("user_id")
  bio         String?      @db.Text
  specialties  Json?        // Array of specialty strings
  type        Json?        // Array of types: ["KOOK", "COURS"]
  city        String?
  address     String?
  experience   String?
  rating       Float       @default(0)
  reviewCount  Int         @default(0) @map("review_count")
  featured     Boolean     @default(false)
  verified     Boolean     @default(false)
  isCompany    Boolean     @default(false) @map("is_company")
  active       Boolean     @default(true)
  stripeAccountId String?  @map("stripe_account_id")
  stripeOnboardingComplete Boolean @default(false)
  @@map("stripe_onboarding_complete")
  createdAt    DateTime   @default(now()) @map("created_at")
  updatedAt    DateTime   @updatedAt @map("updated_at")

  user         User       @relation(fields: [userId], references: [id])
  services     Service[]
  availabilities Availability[]
  bookingsReceived Booking[] @relation("KookerBookings")
  reviewsReceived Review[]  @relation("KookerReviews")
  favoritedBy  Favorite[]
  testimonials Testimonial[]

  @@map("kooker_profiles")
}

// User hosting profile (for hosting services at their place)
model UserProfile {

```

```

    id                Int        @id @default(autoincrement())
    userId            Int        @unique @map("user_id")

    // Address
    address            String?
    addressComplement String?    @map("address_complement")
    city               String?
    postalCode         String?    @map("postal_code")
    country             String?    @default("France")

    // Access
    accessCode         String?    @map("access_code")
    floor              String?
    intercom            String?
    parkingInfo         String?    @db.Text @map("parking_info")

    // Kitchen
    stoveType          String?    @map("stove_type") // "gaz", "induction",
    "électrique", "mixte"
    hasOven             Boolean    @default(false) @map("has_oven")
    hasDishwasher       Boolean    @default(false) @map("has_dishwasher")
    tableCapacity       Int?       @map("table_capacity")
    kitchenNotes        String?    @db.Text @map("kitchen_notes")

    // Dietary preferences
    dietaryRestrictions Json?      @map("dietary_restrictions") // Array
    allergies            Json?      // Array

    // Notes for kooker
    hostingNotes         String?    @db.Text @map("hosting_notes")

    updatedAt           DateTime   @updatedAt @map("updated_at")

    user                User       @relation(fields: [userId], references: [id],
onDelete: Cascade)

    @@map("user_profiles")
}

// Kooker specialty type (reference data)
model KookerSpecialty {
    id                Int        @id @default(autoincrement())
    name              String      @unique
    icon              String?
    createdAt         DateTime   @default(now()) @map("created_at")

    @@map("kooker_specialties")
}

// Service offered by kooker (menu item)
model Service {
    id                Int        @id @default(autoincrement())

```

```

kookerProfileId Int      @map("kooker_profile_id")
title           String
description      String?  @db.Text
type            Json      // ["KOOK"] or ["COURS"] stored as JSON
priceInCents    Int       @map("price_in_cents")
durationMinutes Int       @map("duration_minutes")
minGuests       Int?      @map("min_guests")
maxGuests       Int       @default(1) @map("max_guests")
active          Boolean   @default(true)
allergens       Json?
constraints     Json?
specialty       Json?
prepTimeMinutes Int?      @map("prep_time_minutes")
ingredientsIncluded Boolean @default(false) @map("ingredients_included")
equipmentProvided Boolean @default(false) @map("equipment_provided")
ingredientsList Json?     @map("ingredients_list")
ingredientsBaseServings Int? @map("ingredients_base_servings")
ingredientsPricePerGuestInCents Int?
@map("ingredients_price_per_guest_in_cents")
ingredientsDefaultSource String @default("client")
@map("ingredients_default_source") // "client" or "kooker"
equipmentKooker Json? @map("equipment_kooker")
extraGuestPriceInCents Int? @map("extra_guest_price_in_cents") // For COURS
only
koursDifficulty String? @map("kours_difficulty") // For COURS
koursLocation String? @map("kours_location") // For COURS
createdAt DateTime @default(now()) @map("created_at")
updatedAt DateTime @updatedAt @map("updated_at")

kookerProfile KookerProfile @relation(fields: [kookerProfileId], references:
[id])
images        ServiceImage[]
menuItems     MenuItem[]
bookings      Booking[]
messages      Message[]

@@map("services")
}

// Service images gallery
model ServiceImage {
  id          Int      @id @default(autoincrement())
  serviceId   Int      @map("service_id")
  url         String
  alt         String?
  sortOrder   Int      @default(0) @map("sort_order")
  isCardImage Boolean @default(false) @map("is_card_image")
  createdAt   DateTime @default(now()) @map("created_at")

  service     Service @relation(fields: [serviceId], references: [id],
onDelete: Cascade)

```

```

    @@map("service_images")
}

// Menu item (part of a service)
model MenuItem {
    id                Int          @id @default(autoincrement())
    serviceId         Int          @map("service_id")
    category          String
    name              String
    description        String?
    sortOrder         Int          @default(0) @map("sort_order")

    service           Service      @relation(fields: [serviceId], references: [id],
onDelete: Cascade)

    @@map("menu_items")
}

// Booking - Reservation
model Booking {
    id                Int          @id @default(autoincrement())
    userId            Int          @map("user_id")
    kookerProfileId   Int          @map("kooker_profile_id")
    serviceId         Int          @map("service_id")
    date              DateTime
    startTime          String       @map("start_time") // HH:MM format
    endTime           String?      @map("end_time")
    guests            Int          @default(1)
    totalPriceInCents Int          @map("total_price_in_cents")
    status             String       @default("pending") // "pending", "confirmed",
"awaiting_confirmation", "completed", "cancelled"
    notes             String?      @db.Text
    ingredientsSource String       @default("client") @map("ingredients_source") //
"client" or "kooker"
    stripePaymentIntentId String? @map("stripe_payment_intent_id")
    paymentStatus     String       @default("none") @map("payment_status") // "none",
"pending_authorization", "authorized", "captured", "refunded", "cancelled"
    awaitingConfirmationAt DateTime? @map("awaiting_confirmation_at")
    reminderSentAt1   DateTime? @map("reminder_sent_at_1")
    reminderSentAt2   DateTime? @map("reminder_sent_at_2")
    reminderSentAt3   DateTime? @map("reminder_sent_at_3")
    kookerReminderSentAt1 DateTime? @map("kooker_reminder_sent_at_1")
    kookerReminderSentAt2 DateTime? @map("kooker_reminder_sent_at_2")
    kookerReminderSentAt3 DateTime? @map("kooker_reminder_sent_at_3")
    createdAt         DateTime @default(now()) @map("created_at")
    updatedAt         DateTime @updatedAt @map("updated_at")

    user             User          @relation("UserBookings", fields: [userId], references:
[id])
    kookerProfile     KookerProfile @relation("KookerBookings", fields:
[kookerProfileId], references: [id])
    service           Service      @relation(fields: [serviceId], references: [id])

```

```

    payments      Payment[]
    reviews       Review[]

    @@map("bookings")
}

// Payment tracking
model Payment {
    id              Int          @id @default(autoincrement())
    bookingId       Int          @map("booking_id")
    stripePaymentIntentId String @map("stripe_payment_intent_id")
    type            String       // "authorization", "capture", "refund",
    "cancellation", "transfer"
    amountInCents   Int          @map("amount_in_cents")
    commissionInCents Int?       @map("commission_in_cents")
    stripeTransferId String?     @map("stripe_transfer_id")
    status          String       // "pending", "succeeded", "failed"
    metadata        Json?
    createdAt       DateTime     @default(now()) @map("created_at")

    booking          Booking     @relation(fields: [bookingId], references: [id])

    @@map("payments")
}

// Reviews / Ratings
model Review {
    id              Int          @id @default(autoincrement())
    userId          Int          @map("user_id")
    kookerProfileId Int          @map("kooker_profile_id")
    bookingId       Int?         @map("booking_id")
    type            String       @default("user_to_kooker") // "user_to_kooker",
    "kooker_to_user"
    targetUserId    Int?         @map("target_user_id") // For kooker_to_user reviews
    rating          Int          // 1-5
    comment         String?      @db.Text
    status          String       @default("pending") // "pending", "approved",
    "rejected"
    createdAt       DateTime     @default(now()) @map("created_at")

    user            User         @relation(fields: [userId], references: [id])
    kookerProfile   KookerProfile @relation("KookerReviews", fields:
[kookerProfileId], references: [id])
    booking         Booking?     @relation(fields: [bookingId], references: [id])
    targetUser      User?        @relation("ReviewsReceived", fields: [targetUserId],
references: [id])

    @@map("reviews")
}

// Favorites
model Favorite {

```

```

    id            Int            @id @default(autoincrement())
    userId        Int            @map("user_id")
    kookerProfileId Int          @map("kooker_profile_id")
    createdAt     DateTime       @default(now()) @map("created_at")

    user          User           @relation(fields: [userId], references: [id])
    kookerProfile KookerProfile @relation(fields: [kookerProfileId], references:
[id])

    @@unique([userId, kookerProfileId])
    @@map("favorites")
}

// Availability slots for kookers
model Availability {
    id            Int            @id @default(autoincrement())
    kookerProfileId Int          @map("kooker_profile_id")
    date          DateTime
    startTime     String         @map("start_time") // HH:MM format
    endTime       String         @map("end_time")
    isAvailable   Boolean        @default(true) @map("is_available")
    createdAt     DateTime       @default(now()) @map("created_at")

    kookerProfile KookerProfile @relation(fields: [kookerProfileId], references:
[id])

    @@map("availabilities")
}

// Internal messaging
model Message {
    id            Int            @id @default(autoincrement())
    senderId      Int            @map("sender_id")
    receiverId    Int            @map("receiver_id")
    content        String         @db.Text
    read          Boolean        @default(false)
    kookerRecipientId Int?       @map("kooker_recipient_id") // For filtering by
kooker
    bookingId     Int?           @map("booking_id")           // Associated booking
    serviceId     Int?           @map("service_id")           // Associated service
    createdAt     DateTime       @default(now()) @map("created_at")

    sender        User           @relation("SentMessages", fields: [senderId],
references: [id])
    receiver      User           @relation("ReceivedMessages", fields: [receiverId],
references: [id])
    service        Service?      @relation(fields: [serviceId], references: [id])

    @@map("messages")
}

// System configuration

```

```

model Config {
    id          Int          @id @default(autoincrement())
    key         String       @unique
    value       String       @db.Text // Stored as JSON string
    updatedAt   DateTime     @updatedAt @map("updated_at")

    @@map("configs")
}

// Testimonials for homepage
model Testimonial {
    id          Int          @id @default(autoincrement())
    kookerProfileId Int?     @map("kooker_profile_id")
    authorName   String      @map("author_name")
    authorRole   String?     @map("author_role")
    content      String      @db.Text
    rating       Int         @default(5)
    featured     Boolean     @default(false)
    createdAt    DateTime    @default(now()) @map("created_at")

    kookerProfile KookerProfile? @relation(fields: [kookerProfileId], references:
[id])

    @@map("testimonials")
}

// Password reset tokens
model PasswordResetToken {
    id          Int          @id @default(autoincrement())
    userId      Int          @map("user_id")
    token       String       @unique
    expiresAt   DateTime     @map("expires_at")
    used        Boolean      @default(false)
    createdAt   DateTime     @default(now()) @map("created_at")

    user        User         @relation(fields: [userId], references: [id], onDelete:
Cascade)

    @@map("password_reset_tokens")
}

// Email verification and email change tokens
model EmailToken {
    id          Int          @id @default(autoincrement())
    userId      Int          @map("user_id")
    token       String       @unique
    type        String       // "verify" | "change_email"
    expiresAt   DateTime     @map("expires_at")
    used        Boolean      @default(false)
    createdAt   DateTime     @default(now()) @map("created_at")

    user        User         @relation(fields: [userId], references: [id], onDelete:

```

```

Cascade)

    @@map("email_tokens")
}

// Error logs
model ErrorLog {
    id          Int          @id @default(autoincrement())
    message     String       @db.Text
    stack       String?     @db.LongText
    route       String?
    method      String?
    statusCode  Int?         @map("status_code")
    userId      Int?         @map("user_id")
    createdAt   DateTime     @default(now()) @map("created_at")

    @@map("error_logs")
}

// Page load performance logs
model PageViewLog {
    id          Int          @id @default(autoincrement())
    page        String
    loadTimeMs  Int          @map("load_time_ms")
    userId      Int?         @map("user_id")
    createdAt   DateTime     @default(now()) @map("created_at")

    @@map("page_view_logs")
}

```

DATABASE MODELS

Model	Purpose	Key Fields
User	All platform users	id, email, password (bcrypt), firstName, lastName, phone, avatar, role, isAdmin
KookerProfile	Professional kooker information	userId, bio, specialties (JSON), type (JSON), city, address, rating, reviewCount, verified, stripeAccountId
UserProfile	Hosting information for users	userId, address, city, kitchen details, dietary restrictions, allergies
Service	Offering (KOOK/COURS)	kookerProfileId, title, type, priceInCents, durationMinutes, maxGuests, ingredients info, equipment
ServiceImage	Images for services	serviceId, url, isCardImage, sortOrder
MenuItem	Menu items in service	serviceId, category, name, description

Booking	Reservation	userId, kookerProfileId, serviceId, date, startTime, status, totalPriceInCents, paymentStatus
Payment	Payment audit trail	bookingId, stripePaymentIntentId, type, amountInCents, status
Review	User ratings	userId, kookerProfileId, rating (1-5), comment, status (pending/approved)
Favorite	Bookmarked kookers	userId, kookerProfileId (unique constraint)
Availability	Kooker availability slots	kookerProfileId, date, startTime, endTime, isAvailable
Message	Messaging	senderId, receiverId, serviceId, content, read
Testimonial	Homepage testimonials	kookerProfileId, authorName, content, rating, featured
PasswordResetToken	Password recovery	userId, token, expiresAt, used
EmailToken	Email verification & change confirmation	userId, token (unique), type ("verify" "change_email"), expiresAt, used
Config	System configuration	key, value (JSON)
ErrorLog	Server error tracking	message, stack, route, statusCode, userId
PageViewLog	Performance tracking	page, loadTimeMs, userId

BACKEND - SERVER ROUTES

AUTH ROUTES (/api/v1/auth)

POST /register

- **Rate Limit:** 50 req/15 min
- **Validation:** registerSchema
- **Body:**

```
{
  "email": "user@example.com",
  "password": "string (min 8 chars)",
  "firstName": "string",
  "lastName": "string"
}
```

- **Returns:** User object with kookerProfileId (or null)
- **Status Codes:** 201 (success), 409 (email exists), 400 (validation)

POST /login

- **Rate Limit:** 50 req/15 min
- **Validation:** loginSchema

- **Body:**

```
{
  "email": "user@example.com",
  "password": "string"
}
```

- **Returns:** User object with `kookerProfileId`
- **Sets Cookie:** `token` (`httpOnly`, `secure`, `sameSite:strict`, `maxAge: 2h`)
- **Status Codes:** 200 (success), 401 (invalid credentials), 400 (validation)

POST /logout

- **Returns:** { `success: true`, `data: { message: "..."` } }

GET /me (Protected)

- **Returns:** Current user object with `kookerProfileId`
- **Status Codes:** 200, 401 (unauthorized)

POST /forgot-password

- **Rate Limit:** 5 req/15 min
- **Body:** { `"email": "string"` }
- **Logic:** Generates 32-byte reset token, expires in 1 hour
- **Email Sent:** `sendPasswordResetEmail()`
- **Returns:** Always { `success: true` } (never reveals if email exists)
- **Status Codes:** 200

POST /reset-password

- **Validation:** `resetPasswordSchema`
- **Body:** { `"token": "string"`, `"password": "string (min 8)"` }
- **Logic:** Verifies token, hashes password with `bcrypt` (10 rounds), marks token as used
- **Status Codes:** 200, 400 (invalid/expired token)

GET /verify-email (Email verification after registration)

- **Query:** `?token=<string>`
- **Logic:** Validates `EmailToken` (`type="verify"`, not used, not expired) → sets `emailVerified=true`, marks token as used, issues JWT cookie
- **Redirects:** To `/` or `redirect` param on success
- **Status Codes:** 302 (redirect), 400 (invalid/expired token)

POST /resend-verification (Resend verification email)

- **Rate Limit:** 5 req / 15 min
- **Body:** { `"email": "string"` } (or taken from session if authenticated)
- **Logic:** If account exists and `emailVerified=false` → generates new `EmailToken` (`type="verify"`), sends email
- **Returns:** Always { `success: true` } (never reveals if email exists)
- **Status Codes:** 200

USERS ROUTES — EMAIL CHANGE

GET /api/v1/users/confirm-email-change

- **Query:** `?token=<string>`

- **Logic:** Validates `EmailToken` (type="change_email", not used, not expired) → sets `user.email` = `user.pendingEmail`, clears `pendingEmail`, marks token as used, issues new JWT cookie
 - **Status Codes:** 302 (redirect to profile), 400 (invalid/expired token)
-

KOOKERS ROUTES (`/api/v1/kookers`)

GET `/` (Search with filters)

- **Query Params:**
 - `q` : Full-text search (name, bio, city, specialties, services)
 - `type` : Filter by type (KOOK/COURS) - checks service types
 - `specialty` : Filter by specialty array
 - `city` : Filter by city (database level if no `q`, JS level if `q`)
 - `minPrice` / `maxPrice` : Price range in euros (stored in cents)
 - `difficulty` : Course difficulty filter
 - `page` : 1-based pagination (default 1)
 - `limit` : Items per page (1-50, default 12)
- **Returns:** Paginated kookers with active services, featured/rating order
- **Response:**

```
{
  "success": true,
  "data": {
    "kookers": [...],
    "pagination": { "page": 1, "limit": 12, "total": 100, "totalPages": 9 }
  }
}
```

GET `/dashboard/stats` (Protected, Kooker Required)

- **Returns:**

```
{
  "totalBookings": 42,
  "pendingBookings": 3,
  "totalRevenue": 125000, // in cents
  "avgRating": 4.5,
  "totalReviews": 28
}
```

GET `/:id` (Kooker Profile Detail)

- **Returns:** Complete kooker profile with:
 - user info (firstName, lastName, avatar, phone, createdAt)
 - services (active only) with images, menuItems
 - reviews (user_to_kooker, approved only)
 - availabilities (future only)
 - confirmedSlots (upcoming bookings with status)
- **Status Codes:** 200, 404 (not found), 400 (invalid ID)

POST `/become` (Protected, Create Kooker Profile)

- **Validation:** `becomeKookerSchema`
- **Body:**

```
{
  "bio": "string (optional)",
  "specialties": ["string array"],
  "type": ["KOOK" or "COURS"],
  "city": "string",
  "experience": "string (optional)",
  "isCompany": "boolean (optional)"
}
```

- **Creates:** KookerProfile with active=false
- **Updates:** User role to "kooker"
- **Invalidates:** Auth cache
- **Returns:** KookerProfile object
- **Status Codes:** 201, 409 (already kooker), 400 (validation)

PUT /profile (Protected, Kooker Required)

- **Validation:** `updateKookerProfileSchema`
- **Body:** Any subset of: bio, specialties, type, city, experience, address, isCompany
- **Returns:** Updated KookerProfile
- **Status Codes:** 200, 403 (not kooker), 400 (validation)

SERVICES ROUTES (/api/v1/services)

GET /search (Search services — public)

- **Params:** `q`, `type`, `specialty`, `city`, `minPrice`, `maxPrice`, `difficulty`, `featured`, `date`, `page` (défaut 1), `limit` (défaut 12)
- **Filtres:**
 - Uniquement services `active: true` appartenant à un kooker `active: true`
 - `q` : recherche texte sur titre, description, nom kooker, spécialités
 - `type` : contenu du champ JSON `type` du service
 - `specialty` : spécialités du kookerProfile
 - `city` : sur `kookerProfile.city` (case-insensitive)
 - `minPrice` / `maxPrice` : sur `priceInCents` (euros → centimes)
 - `difficulty` : sur `koursDifficulty` (COURS uniquement)
 - `featured` : `kookerProfile.featured = true`
 - `date` (YYYY-MM-DD) : filtre disponibilité — retourne uniquement les services dont le kooker a une disponibilité ce jour-là
- **Include:** images (isCardImage prioritaire), kookerProfile (id, city, rating, reviewCount, featured, verified, user)
- **Tri:** `kookerProfile.featured DESC` puis `kookerProfile.rating DESC`
- **Réponse:** `{ success, data: { services[], pagination: { page, limit, total, totalPages } } }`
- **Status Codes:** 200

⚠ Cette route est déclarée **avant** `/:id` dans le fichier pour éviter que "search" soit capturé comme un ID.

GET /kooker/:id (Get all services for kooker)

- **Returns:** Array of services with images and menuitems
- **Ordering:** createdAt DESC

GET /:id (Get single service detail)

- **Returns:** Service with images, menuitems, kookerProfile (user details)
- **Status Codes:** 200, 404

POST / (Protected, Kooker Required, Create Service)

- **Validation:** createServiceSchema
- **Body:**

```
{
  "title": "string",
  "description": "string (optional)",
  "type": ["KOOK" or "COURS"],
  "priceInCents": 5000, // e.g., 50 EUR
  "durationMinutes": 120,
  "minGuests": 1,
  "maxGuests": 6,
  "allergens": ["array"],
  "constraints": ["array"],
  "specialty": ["array"],
  "prepTimeMinutes": 30,
  "ingredientsIncluded": false,
  "equipmentProvided": true,
  "ingredientsList": [{ "name": "s", "quantity": "s", "unit": "s" }],
  "ingredientsBaseServings": 4,
  "ingredientsPricePerGuestInCents": 1000,
  "ingredientsDefaultSource": "client",
  "equipmentKooker": ["array"],
  "extraGuestPriceInCents": 500, // For COURS
  "koursDifficulty": "débutant", // For COURS
  "koursLocation": "chez moi", // For COURS
  "menuItems": [{ "category": "s", "name": "s", "description": "s" }],
  "images": ["url1", "url2"]
}
```

- **Creates:** Service + ServiceImages + Menuitems
- **Returns:** Service with images and menuitems
- **Status Codes:** 201, 403 (not kooker), 400 (validation)

PUT /card-image/:imageId (Protected, Kooker Required)

- **Logic:** Sets one image as card image (clears others for this kooker)
- **Returns:** { "imageId": 123 }
- **Status Codes:** 200, 403 (not owner), 404 (image not found)

PUT /:id (Protected, Kooker Required, Update Service)

- **Validation:** updateServiceSchema (all fields optional)
- **Logic:**
 - Verifies ownership

- Replaces menuitems if provided
- Preserves isCardImage flag on images if URL matches
- If no card image previously, first image becomes card image
- **Returns:** Updated service with images and menuitems
- **Status Codes:** 200, 403 (not owner), 404, 400 (validation)

DELETE `/:id` (Protected, Kooker Required)

- **Logic:** Deletes service (cascade deletes images, menuitems via FK)
- **Status Codes:** 200, 403, 404

BOOKINGS ROUTES (`/api/v1/bookings`)

GET `/my` (Protected, Get user's bookings)

- **Returns:** Array of bookings with service, kookerProfile (user details), reviews
- **Ordering:** date DESC
- **Includes:** reviews (user_to_kooker only)

GET `/kooker` (Protected, Kooker Required, Get received bookings)

- **Returns:** Array of bookings for this kooker with user details, service
- **Ordering:** date DESC

GET `/:id` (Protected, Booking owner or kooker only)

- **Returns:** Complete booking with service details, kookerProfile, user
- **Status Codes:** 200, 403 (access denied), 404

POST `/` (Protected, Create Booking)

- **Validation:** `createBookingSchema`
- **Body:**

```
{
  "serviceId": 123,
  "date": "2026-03-01",
  "startTime": "19:00",
  "guests": 6,
  "notes": "optional notes",
  "ingredientsSource": "client" // or "kooker"
}
```

- **Logic:**
 - Validates date is not in past
 - Checks service exists and is active
 - Verifies guests <= maxGuests
 - Auto-calculates price:
 - **COURS:** base + (guests-6) * extraGuestPriceInCents (if >6)
 - **KOOK:** base * guests
 - **Ingredients:** +ingredientsPricePerGuestInCents * guests if ingredientsSource="kooker"
 - Checks kooker has Stripe account + onboarding complete
 - Creates Stripe PaymentIntent (manual_capture)
 - Creates Payment audit record

- **Returns:** { booking, clientSecret }
- **Status Codes:** 201, 400 (validation/past date), 404 (service not found)

POST `/:id/confirm-payment` (Protected, Booking owner)

- **Logic:**
 - Validates date not in past (cancels booking if so)
 - Verifies PaymentIntent status = requires_capture
 - Updates paymentStatus to "authorized"
 - Sends notifications: sendBookingRequestToKooker + system message
- **Returns:** { status: "authorized" }
- **Status Codes:** 200, 400 (wrong payment status), 404

PUT `/:id/confirm-completion` (Protected, Booking owner)

- **Logic:**
 - Requires status = "awaiting_confirmation"
 - Sets status = "completed"
 - Calls executeStripeTransfer() to move funds to kooker
 - Sends emails and system message
- **Status Codes:** 200, 400 (wrong status), 404

PUT `/:id` (Protected, User if pending or Kooker if not completed/cancelled)

- **Validation:** updateBookingDetailsSchema
- **Body:** date, startTime, guests, notes (all optional)
- **Logic:**
 - User can edit only if pending
 - Kooker can edit if not completed/cancelled
 - Recalculates price if guests change (for KOOK type)
 - Sends modification emails with change summary
- **Returns:** Updated booking
- **Status Codes:** 200, 400, 403, 404

PUT `/:id/status` (Protected, Kooker Required)

- **Validation:** updateBookingStatusSchema (status: "confirmed" or "cancelled")
- **Logic:**
 - Validates booking date not in past
 - **If confirmed:** Stripe capture payment
 - **If cancelled:** Stripe cancel/refund
 - **If completed:** Stripe transfer to kooker
 - Sends notifications
- **Status Codes:** 200, 400 (wrong status/past date), 404

PUT `/:id/cancel` (Protected, Owner or Kooker)

- **Logic:**
 - Prevents cancellation if already cancelled/completed
 - Handles Stripe cancellation/refund
 - Sends notifications to both parties
- **Status Codes:** 200, 400, 404

REVIEWS ROUTES (`/api/v1/reviews`)

GET /kooker/:id (Get approved reviews for kooker)

- **Returns:** Array of reviews (type=user_to_kooker, status=approved) with user details
- **Ordering:** createdAt DESC

GET /booking/:id (Protected)

- **Returns:** All reviews for a booking
- **Status Codes:** 200, 401, 404

POST / (Protected, Create user-to-kooker review)

- **Validation:** createReviewSchema
- **Body:**

```
{
  "kookerProfileId": 123,
  "bookingId": 456, // optional
  "rating": 5,
  "comment": "string"
}
```

- **Logic:**
 - Prevents self-review
 - If bookingId: verifies booking is completed, checks no existing review on booking
 - If no bookingId: checks no existing review per user per kooker
 - Creates review with status = "pending" (requires admin approval)
 - Sends sendNewReviewPendingToAdmins()
 - **Rating is NOT updated on profile until admin approves**
- **Returns:** Review object with status="pending"
- **Status Codes:** 201, 400, 409 (duplicate)

POST /kooker-to-user (Protected, Kooker Required)

- **Validation:** createKookerReviewSchema
- **Body:**

```
{
  "bookingId": 456,
  "rating": 5,
  "comment": "string"
}
```

- **Logic:**
 - Verifies booking belongs to this kooker
 - Verifies booking is completed
 - Kooker can rate independently — no prerequisite on client's review
 - Checks kooker hasn't already reviewed this booking
 - Creates review with status = "pending" (admin moderation required)
 - Does NOT affect kooker's rating/reviewCount
 - **Returns:** Review object
 - **Status Codes:** 201, 400, 404, 409
-

FAVORITES ROUTES (/api/v1/favorites)

GET / (Protected)

- **Returns:** User's favorites with kookerProfile (user, active services)
- **Ordering:** createdAt DESC

POST /:kookerId (Protected, Add favorite)

- **Logic:** Upsert (idempotent - return existing if already favorited)
- **Returns:** Favorite object
- **Status Codes:** 201, 404 (kooker not found)

DELETE /:kookerId (Protected)

- **Status Codes:** 200, 404
-

AVAILABILITY ROUTES (/api/v1/availability)

GET /kooker/:id (Get kooker's future availabilities)

- **Returns:** Array of availabilities (date >= today) ordered by date/startTime
- **Status Codes:** 200, 400 (invalid ID)

PUT / (Protected, Kooker Required, Batch upsert)

- **Body:**

```
{
  "availabilities": [
    { "date": "2026-03-01", "startTime": "19:00", "endTime": "22:00",
      "isAvailable": true }
  ]
}
```

- **Logic:**
 - Deletes all future availabilities for this kooker
 - Creates new ones from request
 - **Returns:** Array of newly created availabilities
 - **Status Codes:** 200, 400
-

MESSAGES ROUTES (/api/v1/messages)

GET /unread-count (Protected)

- **Returns:** { count: 5 }

GET /conversations (Protected)

- **Logic:**
 - Loads last 500 messages involving this user
 - Groups by partner + servcId
 - Counts unread per conversation
 - Sorts by lastMessage date DESC
- **Returns:** Array of conversations with user, lastMessage, unreadCount, service

GET /conversation/:userId (Protected, Optional ?servcId query)

- **Logic:**
 - Fetches messages between current and other user (optionally filtered by `serviceId`)
 - Marks all received messages as read
 - Both operations in parallel
- **Returns:** Array of messages ordered by `createdAt` ASC
- **Status Codes:** 200, 400 (invalid `userId`)

POST / (Protected, Send message)

- **Validation:** `sendMessageSchema`
- **Body:**

```
{
  "receiverId": 123,
  "content": "message text",
  "serviceId": 456,
  "kookerRecipientId": 789 // optional
}
```

- **Logic:**
 - Verifies receiver and service exist
 - Creates message
 - Sends email notification (with 5-min cooldown per recipient)
- **Returns:** Message with sender and receiver details
- **Status Codes:** 201, 404 (recipient/service not found), 400

DELETE /conversation/:userId (Protected, Optional ?serviceId query)

- **Logic:** Deletes all messages with `userId` (optionally filtered by `serviceId`)
- **Status Codes:** 200, 400

DELETE /:id (Protected)

- **Logic:** Only sender or receiver can delete
- **Status Codes:** 200, 403, 404

REVIEWS ROUTES - ADMIN (/api/v1/admin/reviews)

GET /reviews (Protected, Admin Required)

- **Query params:** `?status=pending|approved|rejected` , `?type=user_to_kooker|kooker_to_user` (optional), `page` , `limit`
- **Returns:** Paginated reviews of both types with user, kookerProfile, and targetUser details
- **Default:** all types returned if `?type` is omitted

PUT /reviews/:id/status (Protected, Admin Required)

- **Body:** `{ "status": "approved" | "rejected" }`
- **Logic:**
 - **If approved:** `status` → `approved` ; kooker rating recalculated **only for** `user_to_kooker` reviews
 - **If rejected:** review deleted ; kooker rating recalculated **only for** `user_to_kooker` reviews
 - `kooker_to_user` reviews never affect kooker's rating/reviewCount
- **Status Codes:** 200, 400, 404

DELETE /reviews/:id (Protected, Admin Required)

- **Logic:** Deletes review ; recalculates kooker rating **only if** `user_to_kooker`
 - **Status Codes:** 200, 404
-

TESTIMONIALS ROUTES (/api/v1/testimonials)

GET / (Get featured testimonials)

- **Returns:** Array of featured testimonials (featured=true) with kookerProfile
- **Ordering:** createdAt DESC

POST / (Protected, Submit testimonial)

- **Body:** { "authorName": "s", "authorRole": "s", "content": "s", "rating": 5 }
 - **Validation:** content >= 10 chars, rating 1-5
 - **Creates:** Testimonial with featured=false (requires admin to feature)
 - **Status Codes:** 201, 400 (validation)
-

USERS ROUTES (/api/v1/users)

PUT /profile (Protected)

- **Validation:** updateUserProfileSchema
- **Body:** firstName, lastName, phone, email (all optional)
- **Logic:**
 - Checks email uniqueness if changed
 - Re-issues JWT if email changed (to keep cookie in sync)
- **Returns:** Updated user object
- **Status Codes:** 200, 409 (email taken), 400

PUT /avatar (Protected)

- **Body:** { "avatar": "url" }
- **Returns:** Updated user
- **Status Codes:** 200, 400

GET /hosting-profile (Protected)

- **Returns:** UserProfile or null

PUT /hosting-profile (Protected)

- **Validation:** updateHostingProfileSchema
 - **Body:** address, city, postalCode, kitchen details, dietary restrictions, allergies, hostingNotes (all optional)
 - **Logic:** Upsert (create if not exists, update if exists)
 - **Returns:** UserProfile
 - **Status Codes:** 200, 400
-

UPLOAD ROUTES (/api/v1/upload)

POST / (Protected, Upload single file)

- **Middleware:** multer (5MB limit, JPEG/PNG/WebP/GIF only)
- **Logic:** Stores file with `${Date.now()}-${randomString}.ext` naming
- **Returns:**

```
{
  "success": true,
  "data": {
    "url": "/uploads/1234567-abc123.jpg",
    "filename": "1234567-abc123.jpg",
    "mimetype": "image/jpeg",
    "size": 102400
  }
}
```

- **Status Codes:** 201, 400 (no file/wrong type)
-

ADMIN ROUTES (/api/v1/admin)

GET /config/public (Public - no auth needed)

- **Returns:** Configuration for forms (commissionKours, commissionKook, specialties, units, etc.)

GET /stats (Protected, Admin Required)

- **Returns:** { userCount, kookerCount, bookingCount, revenueInCents }

GET /users (Protected, Admin Required)

- **Query:** search, role, page, limit
- **Returns:** Paginated users with kooker profile info

PUT /users/:id (Protected, Admin Required)

- **Body:** role, suspended, isAdmin
- **Returns:** Updated user

DELETE /users/:id (Protected, Admin Required)

GET /kookers (Protected, Admin Required)

- **Query:** search, page, limit
- **Returns:** Paginated kookers with counts

PUT /kookers/:id (Protected, Admin Required)

- **Body:** featured, verified, active
- **Returns:** Updated kooker

GET /reviews/kooker/:id (Protected, Admin Required)

PUT /reviews/:id/status (Protected, Admin Required)

- **Recalculates kooker rating on approval**

DELETE /reviews/:id (Protected, Admin Required)

GET /bookings (Protected, Admin Required)

- **Query:** status, page, limit
- **Returns:** Paginated bookings

GET /services (Protected, Admin Required)

- **Returns:** Paginated services

GET /testimonials (Protected, Admin Required)

PUT /testimonials/:id (Protected, Admin Required)

DELETE /testimonials/:id (Protected, Admin Required)

GET /config (Protected, Admin Required)

- **Returns:** All configuration keys/values

PUT /config/:key (Protected, Admin Required)

- **Body:** { "value": any }
- **Upserts:** Config entry

GET /kpis (Protected, Admin Required)

- **Query:** period (day|week|month)
- **Returns:** Revenue, users, bookings, kookers with delta vs previous period

GET /business-charts (Protected, Admin Required)

- **Returns:** 12-week acquisition, booking status breakdown, top kookers, platform health, recent bookings

GET /tech-stats (Protected, Admin Required)

- **Returns:** DB ping time, table counts, upload folder size, error logs, page timings
-

STRIPE ROUTES (/api/v1/stripe)

POST /webhook (Public - no auth)

- **Signature Verification:** Verifies stripe-signature header
 - **Handles:** payment_intent.succeeded, charge.refunded
-

METRICS ROUTES (/api/v1/metrics)

ZOD VALIDATION SCHEMAS

Auth Schemas (server/src/schemas/auth.ts)

```
registerSchema = z.object({
  email: z.string().email('Email invalide'),
  password: z.string().min(8, 'Le mot de passe doit contenir au moins 8 caracteres'),
  firstName: z.string().min(1, 'Le prenom est requis'),
  lastName: z.string().min(1, 'Le nom est requis'),
});

loginSchema = z.object({
  email: z.string().email('Email invalide'),
  password: z.string().min(1, 'Le mot de passe est requis'),
});

forgotPasswordSchema = z.object({
  email: z.string().email('Email invalide'),
});
```

```

resetPasswordSchema = z.object({
  token: z.string().min(1, 'Token requis'),
  password: z.string().min(8, 'Le mot de passe doit contenir au moins 8 caractères'),
});

```

Booking Schemas (server/src/schemas/booking.ts)

```

createBookingSchema = z.object({
  serviceId: z.number({ required_error: 'serviceId est requis' }),
  date: z.string({ required_error: 'La date est requise' }),
  startTime: z.string({ required_error: "L'heure de debut est requise" }),
  guests: z.number().optional().default(1),
  notes: z.string().optional(),
  ingredientsSource: z.enum(['client', 'kooker']).optional().default('client'),
});

updateBookingStatusSchema = z.object({
  status: z.enum(['confirmed', 'cancelled'], {
    required_error: 'Le statut est requis',
    invalid_type_error: 'Statut invalide',
  }),
});

updateBookingDetailsSchema = z.object({
  date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'Format date invalide (YYYY-MM-DD)'),
  startTime: z.string().regex(/^\d{2}:\d{2}$/, 'Format heure invalide (HH:MM)'),
  guests: z.number().int().min(1).max(200).optional(),
  notes: z.string().max(2000).nullable().optional(),
});

```

Kooker Schemas (server/src/schemas/kooker.ts)

```

becomeKookerSchema = z.object({
  bio: z.string().optional(),
  specialties: z.array(z.string()),
  type: z.array(z.string()),
  city: z.string().min(1, 'La ville est requise'),
  experience: z.string().optional(),
  isCompany: z.boolean().optional(),
});

updateKookerProfileSchema = z.object({
  bio: z.string().optional(),
  specialties: z.array(z.string()).optional(),
  type: z.array(z.string()).optional(),
});

```

```

    city: z.string().optional(),
    experience: z.string().optional(),
    address: z.string().optional(),
    isCompany: z.boolean().optional(),
  });

updateUserProfileSchema = z.object({
  firstName: z.string().optional(),
  lastName: z.string().optional(),
  phone: z.string().optional(),
  email: z.string().email('Email invalide').optional(),
});

updateHostingProfileSchema = z.object({
  address: z.string().optional(),
  addressComplement: z.string().optional(),
  city: z.string().optional(),
  postalCode: z.string().optional(),
  country: z.string().optional(),
  accessCode: z.string().optional(),
  floor: z.string().optional(),
  intercom: z.string().optional(),
  parkingInfo: z.string().optional(),
  stoveType: z.enum(['gaz', 'induction', 'électrique', 'mixte']).optional(),
  hasOven: z.boolean().optional(),
  hasDishwasher: z.boolean().optional(),
  tableCapacity: z.number().int().min(1).max(100).optional(),
  kitchenNotes: z.string().optional(),
  dietaryRestrictions: z.array(z.string()).optional(),
  allergies: z.array(z.string()).optional(),
  hostingNotes: z.string().optional(),
});

```

Service Schemas (server/src/schemas/service.ts)

```

const menuItemSchema = z.object({
  category: z.string(),
  name: z.string(),
  description: z.string().optional(),
});

const ingredientItemSchema = z.object({
  name: z.string(),
  quantity: z.string(),
  unit: z.string(),
});

createServiceSchema = z.object({
  title: z.string().min(1, 'Le titre est requis'),
  description: z.string().optional(),
  type: z.array(z.string()).min(1, 'Au moins un type est requis'),
});

```

```

    priceInCents: z.number().min(0, 'Le prix doit etre positif'),
    durationMinutes: z.number().min(1, 'La duree doit etre au moins 1 minute'),
    minGuests: z.number().optional(),
    maxGuests: z.number().optional().default(1),
    allergens: z.array(z.string()).optional(),
    constraints: z.array(z.string()).optional(),
    specialty: z.array(z.string()).optional(),
    prepTimeMinutes: z.number().optional(),
    ingredientsIncluded: z.boolean().optional(),
    equipmentProvided: z.boolean().optional(),
    ingredientsList: z.array(ingredientItemSchema).optional(),
    ingredientsBaseServings: z.number().int().min(1).optional(),
    ingredientsPricePerGuestInCents: z.number().int().min(0).optional(),
    ingredientsDefaultSource: z.enum(['client', 'kooker']).optional(),
    equipmentKooker: z.array(z.string()).optional(),
    extraGuestPriceInCents: z.number().min(0).optional(),
    koursDifficulty: z.string().optional(),
    koursLocation: z.string().optional(),
    menuItems: z.array(menuItemSchema).optional(),
    images: z.array(z.string()).optional(),
  });

updateServiceSchema = z.object({
  title: z.string().min(1).optional(),
  description: z.string().optional(),
  type: z.array(z.string()).optional(),
  priceInCents: z.number().min(0).optional(),
  durationMinutes: z.number().min(1).optional(),
  minGuests: z.number().optional(),
  maxGuests: z.number().optional(),
  active: z.boolean().optional(),
  allergens: z.array(z.string()).optional(),
  constraints: z.array(z.string()).optional(),
  specialty: z.array(z.string()).optional(),
  prepTimeMinutes: z.number().optional(),
  ingredientsIncluded: z.boolean().optional(),
  equipmentProvided: z.boolean().optional(),
  ingredientsList: z.array(ingredientItemSchema).optional(),
  ingredientsBaseServings: z.number().int().min(1).optional(),
  ingredientsPricePerGuestInCents: z.number().int().min(0).optional(),
  ingredientsDefaultSource: z.enum(['client', 'kooker']).optional(),
  equipmentKooker: z.array(z.string()).optional(),
  extraGuestPriceInCents: z.number().min(0).optional(),
  koursDifficulty: z.string().optional(),
  koursLocation: z.string().optional(),
  menuItems: z.array(menuItemSchema).optional(),
  images: z.array(z.string()).optional(),
});

```

Review Schemas (`server/src/schemas/review.ts`)

```

createReviewSchema = z.object({
  kookerProfileId: z.number({ required_error: 'kookerProfileId est requis' }),
  bookingId: z.number().optional(),
  rating: z.number().min(1, 'La note minimum est 1').max(5, 'La note maximum est 5'),
  comment: z.string().optional(),
});

createKookerReviewSchema = z.object({
  bookingId: z.number({ required_error: 'bookingId est requis' }),
  rating: z.number().min(1, 'La note minimum est 1').max(5, 'La note maximum est 5'),
  comment: z.string().optional(),
});

```

Message Schemas (server/src/schemas/message.ts)

```

sendMessageSchema = z.object({
  receiverId: z.number({ required_error: 'receiverId est requis' }),
  content: z.string().min(1, 'Le contenu du message est requis'),
  kookerRecipientId: z.number().optional(),
  serviceId: z.number({ required_error: 'serviceId est requis' }),
});

```

MIDDLEWARE

Auth Middleware (server/src/middleware/auth.ts)

```

// Auth cache: TTL 60s, avoids 1 DB query per request
const AUTH_CACHE_TTL = 60_000;
const authCache = new Map<number, { data: ...; expiresAt: number }>();

async function authenticate(req, res, next):
  - Retrieves token from cookies
  - Verifies JWT (throws if invalid)
  - Checks cache first (60s TTL)
  - Falls back to DB query: user.findUnique()
  - Implements sliding session:
    * If token age > 30 min: issue fresh token (extends absolute 7-day session)
    * Otherwise: refresh cookie expiry (reset 2h inactivity timer)
  - Sets req.user with: userId, email, role, isAdmin, kookerProfileId
  - Returns 401 on failure

function requireKooker(req, res, next):
  - Checks req.user.kookerProfileId exists
  - Returns 403 "Acces reserve aux kookers" (authenticated but insufficient role)

function requireAdmin(req, res, next):

```

- Checks `req.user.isAdmin === true`
- Returns `403 "Acces reserve aux administrateurs"` (authenticated but insufficient role)

// Note: 401 = not authenticated (no/invalid JWT), 403 = authenticated but wrong role

```
function invalidateAuthCache(userId):  
  - Called after user profile/role changes  
  - Clears cache entry so next request refetches from DB
```

Validation Middleware (`server/src/middleware/validate.ts`)

```
function validate(schema: ZodSchema):  
  - Calls schema.safeParse(req.body)  
  - Returns 400 + fieldErrors if invalid  
  - Sets req.body = result.data if valid  
  - Logs errors to console for debugging
```

Error Handler Middleware (`server/src/middleware/errorHandler.ts`)

```
function errorHandler(err, req, res, next):  
  - If AppError: returns status + message as JSON  
  - Otherwise (unexpected):  
    * Logs to console  
    * Creates ErrorLog entry (fire-and-forget)  
    * Returns generic 500 error
```

Rate Limit Middleware (`server/src/middleware/rateLimit.ts`)

```
function rateLimit(maxRequests: number = 5, windowMs: number = 15*60*1000):  
  - In-memory tracking by IP  
  - Returns 429 if exceeded  
  - Window resets after windowMs
```

Upload Middleware (`server/src/middleware/upload.ts`)

```
multer config:  
  - Storage: diskStorage to /uploads folder  
  - Filename: ${Date.now()}-${random-string}.ext  
  - File filter: JPEG, PNG, WebP, GIF only  
  - Limit: 5MB per file  
  - Error message: "Type de fichier non autorise..."
```

CONFIGURATION & UTILITIES

Env Config (server/src/config/env.ts)

```
export const env = {
  PORT: 3001,
  JWT_SECRET: "...",
  NODE_ENV: "development" | "validation" | "production",
  CORS_ORIGIN: "http://localhost:5173",
  COOKIE_DOMAIN: "localhost",
  RESEND_API_KEY: "re_...",
  APP_URL: "https://dev.weekook.com",
  STRIPE_SECRET_KEY: "sk_test_...",
  STRIPE_PUBLISHABLE_KEY: "pk_test_...",
  STRIPE_WEBHOOK_SECRET: "whsec_...",
};
```

JWT Utilities (server/src/utils/jwt.ts)

```
const JWT_MAX_AGE = '7d'; // Absolute session max
const INACTIVITY_TIMEOUT_MS = 1 * 60 * 60 * 1000; // 1 hour
const TOKEN_REFRESH_THRESHOLD_S = 30 * 60; // Refresh if >30min old

interface TokenPayload {
  userId: number;
  email: string;
  iat?: number; // Issued at
  exp?: number; // Expiration
}

function signToken(payload: { userId, email }): string
  - Signs with HS256
  - Expiry: 7 days

function verifyToken(token: string): TokenPayload
  - Throws on invalid/expired
```

Error Classes (server/src/utils/errors.ts)

```
class AppError extends Error {
  constructor(message: string, statusCode: number = 500)

class UnauthorizedError extends AppError {
  statusCode: 401

class NotFoundError extends AppError {
  statusCode: 404

class ValidationError extends AppError {
  statusCode: 400
```

```
class ForbiddenError extends AppError {  
  statusCode: 403
```

EMAIL SYSTEM

Email Functions (`server/src/lib/email.ts`)

All emails use Resend API + lazy initialization (only creates client if RESEND_API_KEY set).

Email Cooldown: 5 minutes per recipient (in-memory map).

Key Functions:

- **sendPasswordResetEmail**(userEmail, userName, resetUrl)
 - Subject: "Réinitialisation de votre mot de passe Weekook"
 - Link valid 1 hour
- **sendBookingRequestToKooker**(kookerEmail, kookerName, clientName, serviceName, date, startTime, guests, totalPriceInCents)
 - Subject: Nouvelle réservation de \${clientName} – \${serviceName}
 - Info box: Prestation, Date, Heure, Convives, Montant
- **sendBookingConfirmedToUser**(userEmail, userName, kookerName, serviceName, date, startTime, guests, totalPriceInCents)
 - Subject: Réservation confirmée – \${serviceName} avec \${kookerName}
- **sendBookingCancelledToUser**(userEmail, userName, kookerName, serviceName, date)
 - Subject: Réservation annulée – \${serviceName}
- **sendBookingCancelledToKooker**(kookerEmail, kookerName, clientName, serviceName, date)
- **sendBookingModifiedToKooker**(kookerEmail, kookerName, clientName, serviceName, changes: string, bookingId)
 - changes formatted as bullet list
- **sendBookingModifiedToUser**(userEmail, userName, kookerName, serviceName, changes, bookingId)
- **sendPendingReminderToKooker1/2/3(...)**
 - Reminder emails (0h, 24h, 48h after pending)
- **sendBookingExpiredToUser/Kooker(...)**
 - Auto-cancelled after 72h no response
- **sendConfirmationRequestToUser**(userEmail, userName, kookerName, serviceName, date, startTime, bookingId)
 - Request user confirms prestation happened
- **sendConfirmationReminder1/2ToUser(...)**
 - Reminder 24h and 36h after prestation

- **sendAutoConfirmationToUser(...)**
 - Auto-validated after 48h
- **sendCompletionToKooker**(kookerEmail, kookerName, clientName, serviceName, date, totalPriceInCents)
 - Subject: Paiement en cours – \${serviceName}
- **sendNewReviewPendingToAdmins**(reviewerName, kookerName, rating, comment, kookerProfileId)
 - Notifies all admin users
 - Stars display (★☆☆)

FRONTEND ROUTES

Router Configuration (client/src/router/index.tsx)

All pages except login/reset-password are wrapped with **PageLayout** (navbar + footer). Protected routes require authentication + redirect to /connexion if not logged in. Admin routes require authentication + isAdmin=true + redirect to /tableau-de-bord otherwise.

Path	Component	Protection	Purpose
/	HomePage	Public	Hero, featured services (ServiceCard), testimonials, FAQ
/recherche	SearchPage	Public	Search & filter services (ServiceCard) — appel GET /services/search
/kooker/:id	KookerProfilePage	Public	Public kooker profile + services + reviews
/prestation/:id	ServiceDetailPage	Public	Détail d'une prestation : galerie, description, prix, encart kooker, CTA réserver
/tarification	PricingPage	Public	Pricing info
/a-propos	AboutPage	Public	About page
/avantages	BenefitsPage	Public	Benefits page
/confiance	TrustPage	Public	Trust & guarantee page
/faq	FaqPage	Public	FAQ page
/connexion	LoginPage	Public	Login/Register split-screen
/verifier-email	VerifyEmailPage	Public	Email verification after registration
/confirmer-email	ConfirmEmailChangePage	Public	Email change confirmation

/contact	ContactPage	Public	Contact form
/reinitialiser-mot-de-passe	ResetPasswordPage	Public	Password reset form
/tableau-de-bord	UserDashboardPage	Protected	User bookings, favorites
/devenir-kooker	BecomeKookerPage	Protected	Form to become kooker
/kooker-dashboard	KookerDashboardPage	Protected	Kooker stats, bookings, services, planning
/kooker-dashboard/menu/nouveau	CreateMenuPage	Protected	Create new service
/kooker-dashboard/menu/:id/editer	EditMenuPage	Protected	Edit service
/reservation	BookingPage	Protected	Booking confirmation flow
/reservation/:id	BookingDetailPage	Protected	Booking detail + confirm/cancel
/messagerie	MessagesPage	Protected	Messaging interface
/mon-profil	UserProfilePage	Protected	Edit user profile + hosting info
/admin	AdminLayout	Admin Protected	Admin dashboard frame
/admin/utilisateurs	AdminUsersPage	Admin	User management
/admin/kookers	AdminKookersPage	Admin	Kooker management
/admin/reservations	AdminBookingsPage	Admin	Booking management
/admin/services	AdminServicesPage	Admin	Service management
/admin/temoignages	AdminTestimonialsPage	Admin	Testimonials management
/admin/avis	AdminReviewsPage	Admin	Review moderation (pending/approved/rejected)
/admin/faq	AdminFaqPage	Admin	FAQ management
/admin/configuration	AdminConfigPage	Admin	System config
*	NotFoundPage	Public	404 page

API CLIENT & AUTH

API Client (`client/src/lib/api.ts`)

```

const BASE_URL = '/api/v1';

interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: string;
  details?: Record<string, string[]>; // Zod validation errors
}

async function request<T>(method, path, body?): Promise<ApiResponse<T>>
  - Credentials: include (sends cookies)
  - Content-Type: application/json
  - Throws on HTTP error (after parsing response)

export const api = {
  get: <T>(path) => request('GET', path),
  post: <T>(path, body?) => request('POST', path, body),
  put: <T>(path, body?) => request('PUT', path, body),
  delete: <T>(path) => request('DELETE', path),
  upload: <T>(path, formData) => fetch + FormData (no JSON header),
};

```

Auth Context (client/src/contexts/AuthContext.tsx)

```

interface User {
  id: number;
  email: string;
  firstName: string;
  lastName: string;
  phone?: string;
  avatar?: string;
  role: string; // "user", "kooker", "admin", "suspended"
  isAdmin?: boolean;
  kookerProfileId?: number | null;
}

interface AuthContextType {
  user: User | null;
  isLoading: boolean;
  login: (email, password) => Promise<User>;
  register: (data) => Promise<void>;
  logout: () => Promise<void>;
  refreshUser: () => Promise<void>;
}

function AuthProvider({ children }):
  - useEffect: calls refreshUser() on mount
  - setIsLoading(false) after initial check
  - Provides context to children

```

```
export function useAuth():  
  - Returns AuthContextType  
  - Throws if used outside AuthProvider
```

ENVIRONMENT VARIABLES

`.env.example`

```
# Base de données MySQL  
DATABASE_URL="mysql://USER:PASSWORD@HOST:3306/DATABASE_NAME"  
  
# JWT Secret (générer une clé unique par environnement)  
JWT_SECRET="change-me-with-a-strong-secret"  
  
# Port du serveur Express  
PORT=3001  
  
# Environnement  
NODE_ENV=development # development | validation | production  
  
# CORS - URL du frontend  
CORS_ORIGIN=http://localhost:5173  
  
# Cookie domain  
COOKIE_DOMAIN=localhost  
  
# Stripe (paiements)  
STRIPE_SECRET_KEY=sk_test_...  
STRIPE_PUBLISHABLE_KEY=pk_test_...  
STRIPE_WEBHOOK_SECRET=whsec_...  
  
# Resend (emails transactionnels)  
RESEND_API_KEY=re_...  
  
# URL de l'application (utilisée dans les liens des emails)  
APP_URL=https://dev.weekook.com
```

PACKAGE DEPENDENCIES

Root `package.json`

```
{  
  "workspaces": ["client", "server", "shared"],  
  "scripts": {  
    "dev": "concurrently npm:dev:server npm:dev:client",  
    "dev:client": "npm run dev --workspace=client",  
    "dev:server": "npm run dev --workspace=server",  
    "build": "npm run build --workspace=client && npm run build --workspace=server",  
    "db:migrate": "npx prisma migrate dev --schema=prisma/schema.prisma",  
  }  
}
```

```

    "db:seed": "npx tsx prisma/seed.ts",
    "db:push": "npx prisma db push --schema=prisma/schema.prisma",
    "db:generate": "npx prisma generate --schema=prisma/schema.prisma"
  },
  "devDependencies": {
    "concurrently": "^8.2.2",
    "typescript": "^5.4.5",
    "tsx": "^4.7.1",
    "prisma": "^5.11.0"
  },
  "dependencies": {
    "@prisma/client": "^5.11.0"
  }
}

```

Server package.json

Dependencies:

- @prisma/client ^5.11.0 - ORM
- bcrypt ^5.1.1 - Password hashing
- cookie-parser ^1.4.6 - Cookie middleware
- cors ^2.8.5 - CORS middleware
- dotenv ^16.4.5 - Environment variables
- express ^4.18.2 - Web framework
- helmet ^7.1.0 - Security headers
- jsonwebtoken ^9.0.2 - JWT signing/verification
- multer ^1.4.5-lts.1 - File upload
- node-cron ^4.2.1 - Scheduled tasks
- resend ^6.9.3 - Email API
- stripe ^20.4.1 - Payment processing
- zod ^3.22.4 - Input validation

DevDependencies:

- Various @types packages
- tsx ^4.7.1 - TypeScript executor
- typescript ^5.4.5

Scripts:

- npm run dev - Run with tsx watch
- npm run build - TypeScript compilation
- npm start - Run compiled dist/app.js

Client package.json

Key Dependencies:

- react ^18.3.1
- react-router-dom ^6.22.3 - Routing
- @radix-ui/* - 27+ headless UI components
- @stripe/react-stripe-js ^5.6.1 - Stripe integration

- `react-hook-form` ^7.55.0 - Form handling
- `tailwindcss` ^4.0.0 - Styling
- `recharts` ^2.15.2 - Charts
- `sonner` ^2.0.3 - Toast notifications
- `framer-motion` ^11.0.0 - Animations
- `lucide-react` ^0.487.0 - Icons

DevDependencies:

- `vite` ^6.3.5 - Build tool
- `@vitejs/plugin-react-swc` ^3.10.2 - Fast React transformation
- `typescript` ^5.4.5
- `tailwindcss` ^4.0.0

Scripts:

- `npm run dev` - Vite dev server (port 5173)
- `npm run build` - TypeScript + Vite build
- `npm run preview` - Preview production build

Vite Config:

- Plugins: React SWC, Tailwind CSS
- Alias: `@ -> src/`
- Proxy: `/api` and `/uploads` to `http://localhost:3001`
- Build target: esnext

SUMMARY

Weekook V2 is a full-stack booking platform for culinary services using:

- **Frontend:** React 18 + TypeScript + Vite 6 + Tailwind CSS 4 + Radix UI + Stripe.js
- **Backend:** Express.js + Prisma ORM + MySQL
- **Auth:** JWT in httpOnly cookies with sliding sessions (2h inactivity, 7d absolute max)
- **Payments:** Stripe (manual capture, transfers to kooker on completion)
- **Validation:** Zod schemas on both client input validation + server enforcement
- **Email:** Resend API with templates for bookings, password resets, reviews
- **Admin:** Comprehensive dashboard for KPIs, user management, content moderation

31 API endpoints cover:

- Auth (register/login/logout/me/password-reset)
- Kookers (search/profile/become/stats)
- Services (CRUD + images)
- Bookings (full lifecycle with Stripe integration)
- Reviews (user→kooker + kooker→user, admin moderation)
- Messages (internal messaging with email notifications)
- Favorites, Availability, Testimonials
- Admin functions (config, stats, KPIs, charts)

Key Features:

- Rate limiting on auth endpoints
- Auth cache (60s TTL)
- Sliding sessions with token refresh

- Price in cents (stored/computed)
- Duration in minutes
- JSON fields for flexible arrays (specialties, types, allergens)
- Accessibility (Radix UI, keyboard navigation)
- Responsive design (Tailwind responsive utilities)
- Error logging to DB
- Performance page view tracking

End of Technical Specifications Document